# CS 505, Part III: Transaction processing

Victor W. Marek[1]

Department of Computer Science
University of Kentucky

Spring 2014

# What do DBMSes do?

- We learned in the first DB course (CS405 or like) that the language of DBMS consists of three parts:
  - Data definition language
  - Data manipulation language
  - Query language
- SQL *mostly* deals with DDL and QL (but some DML, too)
- Transactions are fundamentally related to data manipulation and this is the part of database language we study here and in the subsequent part of this lecture

# Defining transactions

- A *transaction* is a collection of operations on a database that appears as a unit from the point of view of a user
- For instance, in a bank database a transfer of funds from one account to another is a transaction (e.g. *marek* transfers to *dexter* $100)
- This looks like a single operation, but in reality *two* records are modified. Specifically:
  - The field *balance* in *marek*'s record is decremented by 100
  - The field *balance* in *dexter*'s record is incremented by 100
- From the point of view of bank's business process, such transaction must be executed in entirety or not at all

# But we have to be careful

- In general, we can not predict what transactions will execute when. For instance, possibly, the daily interest is being paid by the bank
- Say, in the middle of this transaction *marek* transfers to *dexter* $100
- On what amount that interest is paid? The one after our transfer completes or before it started?
- The transaction of “paying interest” is interesting because it involves dealing with many records which, for that transaction, will have to be *locked* that is declared unusable by other transactions
- (This specific transaction interferes with normal operations of DBMS (many transactions, potentially, need to wait) and so, normally, such mass operations are executed when the workload of DBMS is minimal)

# Executing transactions

- Transactions are usually initiated by users' programs
  - Those may be written in SQL
  - But likely they are described in a programming language (C++, JAVA, PHP) using connectivity mechanisms (such as ODBC, JDBC)
- SQL allows to set transactions, and defines the language for starting and completing transactions, committing, etc.
- But like in other aspects of interaction of programming languages and SQL there are various ways of handling transactions

# Atomicity

- The requirement that a transaction is executed in its entirety or not at all, is called *atomicity*
- In other words a transaction should behave as an indivisible atom
- But, of course, things happen. A transaction may fail for a variety of reasons
  - The program itself may fail (but some of its instructions already were executed)
  - OS may fail
  - DBMS itself may crash
- If we only take $100 from *marek*'s balance but do not increment *dexter*'s balance then bank's business process fails

# Consistency

- SQL, as we recall from CS405, provides *some* consistency mechanisms
- Examples include: FOREIGN KEY constraints (i.e. referential integrity), UNIQUE constraints (candidate key constraint) and CHECK constraints
- Such constraints can be declared when a table is defined, or added later
- But there may be additional, application dependent, constraints. Examples follow.

# Consistency, cont'd

- Here are examples of user-defined constraints (which users?)
  - The numerical sum of balances of records involved in the transaction must be same before and after the transaction
  - The sum of balances of all clients' accounts must be bigger than a prespecified value
- This maintenance of user-defined constraints is called *consistency of transactions*. That is, the user may impose additional constraints on transactions

# Isolation

- When the transactions are executed, it should appear that one transaction has nothing to do with another
- That is the effect of running (say) two transactions $T_1$ and $T_2$ should be like we executed them one-after-another
- But we can not predict in what order are transactions executed. We normally expect that the resulting picture should be like they executed in the order of submission
- Requirement that the transactions execute without interleaving is *too restrictive*. For instance if both $T_1, T_2$ only read *marek*'s balance (but do not modify it) and one of them does something to *dexter* account, and another to *colin*'s account then it does not matter if we start $T_2$ while $T_1$ is in progress
- The property that the execution *should look* like transactions were done one after the other is called *isolation*

# Durability

- Once a transaction successfully completes, its results should persist
- Once *marek* transfered to *dexter* $100 and that transaction completed successfully, the effects should persist through malfunction
- (We do not claim a malfunction will occur)
- (Of course it may happen that we decide that, for whatever reasons, *dexter* should return $100 to *marek*, but that would be another transaction)

# Durability, cont'd

- Thus if the DBMS crashes after a transaction *T* completed, then when the system restores, the state of the DBMS should reflect the fact that transaction *T* completed
- (But but what does it mean transaction completed? In the main memory? or results written to disk?)
- This property is called *durability* and is required for the integrity of the business process

# ACID property

- The four properties listed above (atomicity, consistency, isolation, durability) are called **ACID** properties
- Transaction processing is supposed to satisfy these properties
- (It may be of interest to know that A, I, and D can not be ensured together in distributed DBMS, in particular “clouds”)

# What did we learn?

- When data is manipulated, a number of natural postulates (abbreviated ACID) arises
- Those are not just theoretical constructs, but rather are entailed by the business requirements
- Transactions occur in various computer science applications, in particular e-commerce
- (In fact tools for e-commerce, in particular from JAVA, support transactions)

# Modeling transactions

- This needs to be remembered always (and in fact is the main legacy of this lecture): **Data "lives" on disk, but is being processed in main memory**
- In principle, this dichotomy is not observed by the casual user, but is very real and drives this section of our lecture
- Accessing a piece of data (a.k.a. *data item*) $X$: $\mathrm{read}(X)$
- Returning that item (possibly updated): $\mathrm{write}(X)$
- We think about reading as copying from the disk to a buffer (a chunk) of main memory
- We think about writing as copying back the updated item to place from which it was copied
- (In reality writing does not occur instantly, and it may even be executed much later)

# Our favorite transaction



- $T$ :
  read($A$);
  $A := A - 100$;
  write($A$);
  read($B$);
  $B := B + 100$;
  write($B$);
- But what about this one?
- $T'$ : read($A$);
  read($B$);
  $A := A - 100$;
  $B := B + 100$;
  write($A$);
  write($B$);
- Same effect?

# What about ACID in our transaction?

- Say, consistency condition is that the value of $A + B$ is an invariant
- Consistency: what if a failure occurs right after line 3? Database could be in an inconsistent state!
- Atomicity: In this example some results are written to non-volatile storage, but transaction did not complete!
- Isolation: If another transaction reads $A, B$ in the middle of $T$, say after line 4, it reads incorrect data
- To ensure isolation modern DBMSes have *concurrency control system* (a.k.a. *CC component*)
- Durability: If we want to guarantee durability then
  (a) Either updates carried out by a transaction are written to to the disk before the transaction completes, or
  (b) Information about the those updates and written to the disk must be sufficient to reconstruct the updates when the DBMS is restarted after the failure
- Recovery system handles both (a) and (b)

# Classifying storage

- *Volatile storage*: one that does not survive crashes. For instance main memory or cache
- Access to volatile memory is fast
- (There are databases that live for long time and in main memory only, but only for very small databases and only for special types of applications)
- *Non-volatile storage*: one that survives system crashes. For instance disks, flash storage, tape, optical media
- Some of these are used for archival storage
- Non-volatile storage is slower than the volatile one. While it is non-volatile it is not permanent: disks die, tapes get demagnetized, flash memory gets corrupted
- *Stable storage*: never dies. (There is no stable storage, but but we simulate it to an extent, for instance using RAID)

# What did we learn?

- Databases “live” on disk, but are processed in main memory
- The effect of this is that when main memory crash then we have to eliminate the effects and this may involve accessing the disk
- There are various types of storage

# States of a transaction



- Transaction do not necessarily complete its execution
- (This is a fundamental issue in business databases, to see the opposite situation think what happens when search engine, for whatever reason, does not display a metadata of a page, or presents it but with a very low priority)
- When a transaction does not complete we call it **aborted**
- Changes made by an aborted transaction **must** have no effect on the state of the database (i.e. can not propagate to the disk)

# States of a transaction, cont'd



- When those changes disappear, transaction is **rolled back**
- Roll back (a.k.a. rollback) is performed by the recovery system
- To facilitate rollback (and possibly redoing transactions), the transaction manager maintains the **log** of updates
- Log consists of log records. These records contain the following information: The id of transaction responsible for the update, the id of the data item that is modified, the value before the update, and the value after the update

# States of a transaction, cont'd

- Thus, the log doubles (approximately) the space used by the database
- (Data once written on the log which is stored, and again written on the disk)
- (This is the price to be paid for integrity guarantees)
- A transaction that completed its execution is called **committed**
- Once the transaction is committed its effects persist. Thus the only way to eliminate its effects is to run a *compensatory transaction*
- (This is, of course, not always possible because other transactions may affect the data)
- (Think about *dexter* spending all his money even though he needs to give *marek* his money back)

# Terminology of transactions



- A transaction is
  - *active* - if it started and stays in this state while executing
  - *partially committed* - after executing its last action
  - *failed* - if the normal execution no longer proceeds
  - *aborted* - once the transaction is rolled back, and the data is restored to the state before the transaction started
  - *committed* - if all the actions executed and the results are written to the non-volatile storage
  - *terminated* - if it is aborted or committed

# What do we do with failed transactions?



- It must be rolled back, Then what?
  - They can be *restarted* (if the failure the result of hardware failure or of software error but that error is not related to the transaction code)
  - (For instance OS failure is software failure of this kind)
  - A restarted transaction is treated as a **new** transaction
  - (The issue here is that, in principle, the transactions should be executed in the order they are submitted. The key issue is “in principle”, see below)
  - Or it can be *killed* (if the failure results from some exception, say an attempt to divide by 0)
  - Killed transactions disappear from the queue

# Observable external writes

- Those are: writes to the screen, writes to files, emails etc.
- Occurring when DBMS is used in an embedded hardware/software systems such as ATM, back-end Web applications etc.
- Generally, DBMSes allow for such writes to occur **only** after commitment but this is not always possible or may have side effects
- Think about ATM where crash could occur after commitment but before money is disbursed
- Specifically, what to do in such situation?
- Similar situations occur in e-commerce
- Compensatory transactions may be necessary in such cases

# What did we learn?

- We have a vocabulary for states of transactions
- Transactions may fail, and this may compromise the integrity of DBMS
- Modern DBMSes allow for *side effects*

# Transaction isolation

- While we want the isolation of transactions (i.e. the *illusion* that transactions are executed serially), for performance reasons we want to *interleave* transactions
- Generally we want to increase *throughput*, i.e. the average number of transactions per unit of time
- Here are the possibilities
  - I/O work, CPU work can be done in parallel
  - Newer processors are multi-core - so various jobs can be done on a single processor in parallel
  - Multiprocessor architectures (such as MPI) and stronger yet (Google map-reduce) schemes have been proposed

# Transaction isolation, cont'd

- Our goal now is to *interleave* transactions, but in such a way that the user will perceive the execution as *serial*
- This will be formally defined later, now let us look at an example
- Let us look at the following two transactions:

| $T_1$ | $T_2$ |
|---|---|
| $\text{read}(A)$; | $\text{read}(A)$; |
| $A := A - 100$; | $\text{new} := A * 0.05$; |
| $\text{write}(A)$; | $A = A - \text{new}$; |
| $\text{read}(B)$; | $\text{write}(A)$; |
| $B := B + 100$; | $\text{read}(B)$; |
| $\text{write}(B)$. | $B = B + \text{new}$; |
| | $\text{write}(B)$. |

- The first one is the familiar transfer of $100 from *marek* to *dexter*
- The second one is . . .

# Serial executions

- Actually, there are two possible serial executions: $T_1 T_2$ and $T_2 T_1$
- Let us assume *marek* balance is \$900, *dexter* balance \$1000
- After the execution $T_1 T_2$ *marek* has \$760, *dexter* \$1140
- After the execution $T_2 T_1$ *marek* has \$755, *dexter* \$1145

# Serial executions, cont'd



- Here is the execution $T_1 T_2$

$T_1$ : read($A$);
$T_1$ : $A = A - 100$;
$T_1$ : write($A$);
$T_1$ : read($B$);
$T_1$ : $B = B + 100$;
$T_1$ : write($B$);
$T_2$ : read($A$);
$T_2$ : new $:= A * 0.05$;
$T_2$ : $A = A -$ new;
$T_2$ : write($A$);
$T_2$ : read($B$);
$T_2$ : $B := B +$ new;
$T_2$ : write($B$);

# But here is another execution

$T_1$ : read($A$);
$T_1$ : $A = A - 100$;
$T_1$ : write($A$);
$T_2$ : read($A$);
$T_2$ : new := $A * 0.05$;
$T_2$ : $A = A -$ new;
$T_2$ : write($A$);
$T_1$ : read($B$);
$T_1$ : $B = B + 100$;
$T_1$ : write($B$);
$T_1$ : commit;
$T_2$ : read($B$);
$T_2$ : $B = B +$ new;
$T_2$ : write($B$);

$T_2$ : commit

- (I wrote the schedule in a single column, not two as in the book, it is just syntactic sugar)
- The result is the same as in $T_1 T_2$

# Schedules

- A *schedule* of transactions $T_1, \ldots T_k$ is listing of operations of $T_1, \ldots, T_k$ in which every transaction is listed in its order
- (For instance execution on the previous slide is a schedule of $T_1$ and of $T_2$
- The effect of schedule is the contents of its variables after the schedule is completed
- Transactions of a schedule are transactions mentioned in that schedule

# Schedules, cont'd



- A schedule is *serial* if every transaction is a segment of the schedule
- A schedule *S* is called *serializable* if its effect is the same as that of *some* serial schedule of transactions of that schedule
- A delicate aspect: we do *not* know in what order transactions were submitted, and we can not impose the order on serial schedule
- (But see below when we talk about time-stamping)

# What did we learn?

- Transactions can be interleaved
- In fact we want to interleave transactions to increase throughput
- We can not predict the order in which transactions will be submitted

# Can we predict what will transactions do?



- Not really - transactions alter values of data items according to user's programs, not our wishes
- But we can, on the basis of what is read and what is written, determine if actions of transactions can be swapped
- Generally, the problems are *only* with actions pertaining to the same data item. That is, if actions pertain to *different* data items then there is no problem with swapping

# Problems with prediction, cont'd

- But then, the conflicts appear in the following situation: two transactions deal with the same data item
- Clearly, before a transaction does anything with an item, say *A*, it has to read it
- Likewise, after it is done with it and altered it, it will write it
- Our goal for now is to see what can be said on the basis of reads and writes

# How read and write actions conflict?

- Say, we have two transactions $T$ and $T'$ and they both deal with an item $A$
- (We look now only at reads and writes)
- If $T$ has line $I$ which is $T : \mathrm{read}(A)$ while $T'$ has line $J$ which is $T' : \mathrm{read}(A)$, then these two lines are *not* in conflict; if we swap them same thing is read (as long as there is no $\mathrm{write}(A)$ in between)
- But what if $I$ is $T : \mathrm{write}(A)$ while $J$ is $T' : \mathrm{read}(A)$?
- There is a potential conflict because what is read by $T'$ after swap may be different of what it was before swap!
- The same situation is when $I$ is $T : \mathrm{read}(A)$ while $J$ is $T' : \mathrm{write}(A)$
- There is again potential conflict

# How read write actions conflict, cont'd



- Finally, when $I$ is $T : \text{write}(A)$ while $J$ is $T' : \text{write}(A)$
- There is, of course, again a conflict, $T$ - which may read $A$ many times, may read a wrong value of $A$
- Thus only in Case 1 (read/read) there is no potential for conflict
- Knowing how the conflicts look like, we would like to be able to answer the following questions:
  - If you give me a schedule $S$, can I, just by looking at read and writes of transactions that are in that schedule tell you that, for sure, that schedule $S$ is equivalent to a serial schedule? If so, which one?
  - If I have a serial schedule, can I swap some of read and writes and still guarantee that the schedule is serializable?

# Let us look at examples

- We have these two transactions (stripped of details)

| | | | |
|---|---|---|---|
| $T_1$ | read($A$); | $T_2$ | read($A$); |
| | write($A$); | | write($A$) |
| | read($B$); | | read($B$); |
| | write($B$). | | write($B$). |

- How could I interleave them?

# Example, cont'd

- Here is one interleaving:

| $T_1$ | $T_2$ |
|---|---|
| read($A$); | |
| write($A$); | |
| | read($A$) |
| | write($A$) |
| read($B$); | |
| write($B$); | |
| | read($B$) |
| | write($B$) |

# Example, cont'd

- Here is another interleaving:

| $T_1$ | $T_2$ |
|---|---|
| read($A$); | |
| write($A$); | |
| | read($A$) |
| read($B$); | |
| | write($A$) |
| write($B$); | |
| | read($B$) |
| | write($B$) |

# What did we learn?

- Instructions of transactions may be in conflict
- Swapping such instructions may result in incorrect execution
- Thus we have to be careful when we interleave transactions

# Conflicting instructions



- We say that two instructions, $I$ and $J$ are *in conflict* if they are operations on same data item, of different transactions, and at least one is write operation.
- So here:

| $T_3$ | $T_4$ |
|---|---|
| read($A$); | |
| | write($A$) |
| write($A$); | |

First and the second instructions are in conflict, as well as the second and third are also in conflict

# Conflicting instructions, cont'd



- If we executed $T_3 T_4$ then the value in $A$ would be that of line 2 (but in this schedule it is value of line 3)
- And if we executed $T_4 T_3$ then $T_3$ reads incorrect value!
- In the previous example ($T_1$, and $T_2$) we could swap, but not here. Why?

# Conflicting instructions, cont'd



- The point is that our schedule is not *conflict equivalent* to a serial schedule
- It is natural to propose the following definition: A schedule *S* is *conflict serializable* if it is conflict equivalent to a serial schedule
- One proves that conflict serializable schedule is serializable

# But is it algorithmic?

- The first question we investigate is whether we can test if a schedule is conflict serializable
- We can, the idea is to reduce the problem to a graph problem
- We assign to a schedule $S$ (only read/write operations, nothing else, no commit) a graph $G_S$ as follows:
  - The vertices of $G_S$ are transactions in $S$
  - We introduce an edge from $T$ to $T'$ if
    - For some item $Q$, $T : \text{write}(Q)$ occurs in $S$ before $T' : \text{read}(Q)$, or
    - For some item $Q$, $T : \text{read}(Q)$ occurs in $S$ before $T' : \text{write}(Q)$, or
    - For some item $Q$, $T : \text{write}(Q)$ occurs in $S$ before $T' : \text{write}(Q)$

# Example

- Let us look at our first serial schedule. Then there there were two transactions, just one edge $T_1 \longrightarrow T_2$
- (Clearly, when we have a serial schedule, then the corresponding graph can be completed to a chain)
- But now, let us look at this schedule

# Example, cont'd

| $T_1$ | $T_2$ |
|---|---|
| read($A$); | |
| | read($A$) |
| | write($A$) |
| | read($B$) |
| write($A$); | |
| read($B$); | |
| write($B$); | |
| | write($B$) |

# Example, cont'd

- (We saw it before)
- Then because of lines 2 and 5 there is an edge from $T_2$ to $T_1$
- But because of lines 7 and 8 there is an edge from $T_1$ to $T_2$
- Thus the graph $G_S$ (here $S$ is our schedule) has a cycle

# Recall topological sorting

- Given a digraph $G$ topological sorting of $G$ is a linear ordering extending $E$ (the set of edges)
- That is, the ordering $\prec$ so that whenever $v \longrightarrow w$ then $v \prec w$
- We should recall from Discrete Math course (or from Data Structure course) that a graph possesses a topological sorting if and only if it has no cycles
- Of course it is very easy to test graph for existence of topological sort, and find such sorting when one exist
- (There may be more than one topological sorting of $G$)
- Any topological sorting of $G_S$ determines a serial schedule conflict equivalent to $S$
- Therefore, $S$ is conflict serializable if and only if $G_S$ is topologically sortable

# Now, the other direction

- Say, we have a serial schedule $S$ consisting of several transactions one after the other
- Then we can “push” operations of later transactions “upward” *as long as we do not introduce new conflicts*
- That is, as long as we *preserve* the graph $G_S$

# Other direction, cont'd

- Example: Say, we have our first example of a transaction

| $T_1$ | $T_2$ |
|---|---|
| read($A$); | |
| write($A$); | |
| read($B$); | |
| write($B$); | |
| | read($A$) |
| | write($A$) |
| | read($B$) |
| | write($B$) |

# Pushing it up

- We push line 5 before line 4

| $T_1$ | $T_2$ |
|---|---|
| read($A$); | |
| write($A$); | |
| read($B$); | |
| | read($A$) |
| write($B$); | |
| | write($A$) |
| | read($B$) |
| | write($B$) |

# Pushing it up, cont'd



- We push line 5 before line 4
- We can do this because the graph did not change!
- Keep pushing!

| $T_1$ | $T_2$ |
|---|---|
| read($A$); | |
| write($A$); | |
| | read($A$) |
| | write($A$) |
| read($B$); | |
| write($B$); | |
| | read($B$) |
| | write($B$) |

- What did we push, and what was achieved?

# What did we learn?

- By looking only at read/write operations we can test a transaction for one type of serializability
- It is based on a natural graph concept, namely topological sorting
- This technique can be used for creating serializable schedules out of serial schedules
- Such schedules achieve a better throughput

# Unrecoverable failures



- So far we looked at read and write operations. But commit needs to be considered, too
- Let us look at this example:

| $T_1$ | $T_2$ |
|---|---|
| read($A$); | |
| write($A$); | |
| | read($A$) |
| | read($C$) |
| | commit |
| read($B$) | |
| abort | |

- As $T_1$ failed, it will be rolled back
- But when $T_1$ is rolled back, $T_2$ read *wrong* value of $A$

# Unrecoverable failures, cont'd

- But the transaction $T_2$ committed
- We would have to issue an compensatory transaction!
- What was the problem? In our partial schedule, $T_2$ committed while $T_1$ was active
- $T_2$ is *dependent* on $T_1$ but committed before $T_1$
- Such schedule is called *unrecoverable* schedule
- Certainly it is inappropriate, as it can result in problems (of course, maybe $T_1$ does not fail at line 7, but as we schedule, we do not know)

# Unrecoverable failures, cont'd



- Thus the following definition makes sense:
- A schedule $S$ is *recoverable* if for each $T, T'$ such that $T'$ reads a data item written by $T$, $T$ must commit before $T'$ commits
- In our example the line 5 would have to be executed *after* $T_1$ commits
- Thus, when $T_1$ fails, $T_2$ is also rolled back without the need for compensating transaction

# Cascade-less schedules

- In our example we had cascading rollback. Things may get worse
- Let us look at an example

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| read($A$); | | |
| write($A$); | | |
| | read($A$) | |
| | write($A$) | |
| | | read($A$) |
| | | write($A$) |
| abort | | |

- This partial schedule results in rolling back all three transactions
- Of course, $T_3$ depended on $T_2$ which, in turn, depended on $T_1$
- When $T_1$ is rolled back, transactions that depended on it ($T_2, T_3$ in our case) have to be rolled back as well

# Cascade-less schedules, cont'd



- This situation is called *cascading rollback* and is, obviously undesirable
- A *cascade-less schedule* is one where there is no possibility of cascading rollbacks if there is a failure
- Formally, a cascade-less schedule is one with the following property: for each pair of transactions $T$, $T'$, if $T'$ reads an item written by $T$, then *commit* operation of $T$ must occur before the *read* operation of $T'$
- One can prove that every cascade-less schedule is recoverable

# What did we learn?

- Transaction failure may result in rollback of more than one transaction
- This is undesirable
- There are conditions on schedules that prevent cascading failures

# Transaction isolation levels

- Serializability is a desired property of schedules
- But the protocols for ensuring serializability are strict and costly
- That is, maintaining serializability lowers the throughput
- For that reason SQL allows for a language that weakens requirement of serializability
- The problem is with long transactions, esp. those involving e-commerce and social networks
- Also, in some applications (e.g. Google) the results of long transactions do not need to be precise
- To standardize the types of transactions, SQL introduces *isolation levels*
- Actually four such levels are introduced
- (In decreasing order of closeness to serializable transactions)

# Transaction isolation levels, cont'd



- *Serializable*. In principle, what we studied above, but in reality not always serializable
- *Repeatable read*. This level allows only committed data to be read and additionally requires that between two read operations of a given data item $A$ by a given transaction $T_i$ no *other transaction* can update $A$
- However, the transaction does not have to be serializable w.r.t. other transactions. For instance when searching for data satisfying some condition a transaction may find some data inserted by the same transaction that updated $A$ (thus phantom records may occur)

# Transaction isolation levels, cont'd



- *Read committed*. Allows only committed data to be read, but does not require repeatable read. Thus if some other transaction changes some field in a record *R* and commits, you will see different values second time you read *R* (again a variation on phantom phenomenon)
- *Read uncommitted*. Allows uncommitted data to be read
- (Yes, this lowest level is sometimes useful, for instance when looking on the "ticker" data on the stock exchange)
- All levels disallow *dirty write* that is does not allow to write a data item that was previously written (by another, uncommitted transaction)

# Transaction isolation levels, cont'd



- SQL default level (but this is not enforced) is *read committed*
- SQL has the language (but again, not enforced) for other levels of isolation
- For instance ORACLE supports this language:

SET TRANSACTION LEVEL SERIALIZABLE

- (But DB2 has a completely different language for the same thing)
- Changing isolation level of a transaction can be done only once, and at the beginning
- Thus the designer of the transaction (i.e. the programmer) must understand the process
- Moreover even if the programmer sets up the level of isolation, DBMS has an option not to enforce it

# Serializablility in real DBMSes

- Non-repeatable reads happen in real world
- This takes place often in Web applications such as e-commerce
- For instance you reserve a seat on a plane but before you grab your credit card and finish the checkout - it is gone
- The reason is that this type of transactions is very long from the point of view of databases supporting reservation system and, generally, the uncertainty about the client's intentions makes it impossible to lock the corresponding records
- Thus someone else may have reserved the seat in the meanwhile
- (B.t.w. it only looks like there are different reservation systems like Expedia, Kayak, etc. In reality it is the same back-end system)
- Similar phenomena occur in other reservation systems, say concert tickets, auctions of multiple objects, etc.

# Implementing isolation levels

- Need for *concurrency-control policies*
- Two prevailing types of such policies:
  - Lock-based
  - Time-stamp based

# Implementing isolation levels, cont'd



- Locking is a technique where a transaction limits access of other transactions to data
- There are various types of locks (exclusive, shared) - to be discussed later
- Time-stamp assigns to a transaction its beginning in time
- (We will discuss it extensively below)
- Th idea is to create illusion that transactions are processed in the order of their submission
- For each data item *A* we maintain the data on the most recent time-stamp of transaction that *read A* and also time-stamp of most recent recent transaction that *wrote A*
- When transactions create *deadlock* because of conflicts on various data items, we roll back some of transactions

# What did we learn?

- While we want full isolation of transactions we may settle for less
- Four basic isolation levels: serializability, repeatable reads, read committed, read uncommitted
- Various techniques are used for implementation of isolation
- Isolation levels used for fine-tuning of performance

# Snapshot isolation, the idea

- The idea is to have transactions to work on database in parallel
- In this approach each transactions gets its own copy of a database
- Of course we do this dynamically, duplicating only the part of DB that the transaction in question deals with
- Then, when a transaction *T* partially commits (i.e. does all its work *on its own copy*) it goes into committed state *only* if no other concurrent transaction modified the data that *T* intends to update
- Transactions that can not commit in this way - abort and get restarted (but then eventually work on a different data!)
- Here are the benefits:
  - We can use OS for scheduling
  - We never wait (but run the risk of aborting)
  - Read-only transactions never abort

# The idea of snapshot isolation, cont'd



- On SQL systems most transactions SELECT, thus read-only
- But if $T$ reads data that $T'$ updates, and $T'$ reads data (different data) that $T$ updates, we run risk of inconsistent state of database
- Nevertheless parallelism inherent in snapshot isolation is very attractive from the efficiency point of view
- Various real-world DBMSes offer the option of snapshot isolation

# Phantom records, what is it?

- So far we assumed that the database does not change, i.e. there are no inserts or deletes
- But in reality when a transaction $T$ processes a table, another transaction $T'$ may insert a new record into that table or eliminate a record from that table
- Generally, this creates a conflict between $T$ and $T'$

# Phantom records, cont'd



- Generally, we should not allow for modifications to the table that is being processed by a transaction because, for instance when a record is added, it may be satisfying SELECT query but not shown in the answer
- Such record is called *phantom record*
- There is a way of handling this situation, by means of *index locking* (so changes can not be executed when a transaction processes a table)
- But this is a costly solution and often not used at all

# What did we learn?

- There is a lot to transaction processing
- Concurrency control is costly and for that reason often disregarded (to an extent)
- There are various policies to enforce concurrency
- Database “lives” and this creates additional problems

# Locks, their classification

- The idea of locks: data is accessed by transactions in a mutually exclusive manner
- One way to do so is to assign locks (on data items) to transactions
- The idea is: 'When a transaction $T$ accesses a data item $A$ then no other transaction $T'$ can *modify* $A$'
- (But the same applies to $T'$ - so this imposes a restriction on $T$ as well)
- There are various kinds of locks. Here we consider two:
  - *Shared* lock. If $T$ obtained shared-mode lock on $A$ (abb. $lock - S(A)$) then $T$ can read, but it can not modify $A$
  - *Exclusive* lock. If $T$ obtained exclusive-mode lock on $A$ (abb. $lock - X(A)$) then $T$ can both read and modify $A$

# Colliding locks

- Say, we have transactions $T$, $T'$ and both need to access an item $A$
- Here is their *compatibility matrix*

| | S | X |
|---|---|---|
| S | 1 | 0 |
| X | 0 | 0 |

- Compatibility matrix depends on transactions and on data item
- This matrix is used by *concurrency control manager* to assign locks
- Specifically, each transaction *requests* a lock before it is granted
- When the request on locking some data $A$ is made, CC manager (actually its component, lock manager) checks the request against all other locks on data item $A$ already granted
- If the matrices allow for the grant, it is given; otherwise, if at least one matrix disallows the grant - it is not given

# Granting locks process

- Specifically:
  - When a transaction $T$ requests $lock - S(A)$ then if previously CC manager granted no locks on $A$ or only $lock - S(A)$ was granted then $T$ will receive $lock - S(A)$ and may proceed. But if some other transaction was already granted $lock - X(A)$ then $lock - S(A)$ is *not* granted
  - When a transaction $T$ requests $lock - X(A)$ then if previously *any* lock on $A$ was granted (and not released) then the request is not granted
- But then, when the grant is not given, what do we do with $T$? We have to wait until the lock or locks (there may be several such locks if these locks are $lock - S(A)$) are released
- This means we may face *deadlock*, and indeed it is possible that deadlock occurs

# But we have to be careful

- Let us look at an example

| $T_1$ | $T_2$ |
|---|---|
| $lock-X(A)$; | $lock-S(B)$; |
| $read(A)$; | $read(B)$; |
| $A := A - 100$; | $unlock(B)$; |
| $write(A)$; | $lock-S(A)$; |
| $unlock(A)$; | $read(A)$; |
| $lock-X(B)$; | $C = A + B$; |
| $B := B + 100$; | $unlock(A)$; |
| $write(B)$; | $display(C)$; |
| $unlock(A)$; | |

- $T_1$ is a familiar transaction (which one?)
- $T_2$ displays $C$, the sum of variables $A + B$
- Say we start with $A == 300$, $B == 200$
- Under the executions $T_1T_2$ and $T_2T_1$, $C$ takes value 500

# Let us look at a specific schedule

(In the third column we display actions of CC-mgr)

| $T_1$ | $T_2$ | $CC - mgr$ |
|---|---|---|
| $lock - X(A)$ | | |
| | | $grant - X(A, T_1)$ |
| $read(A)$ | | |
| $A := A - 100$ | | |
| $write(A)$ | | |
| $unlock(A)$ | | |
| | $lock - S(B)$ | |
| | | $grant - S(B, T_2)$ |
| | $read(B)$ | |
| | $unlock(B)$ | |
| | $lock - S(A)$ | |
| | | $grant - S(A, T_2)$ |
| | $read(A)$ | |
| | $unlock(A)$ | |
| | $C := A + B$ | |
| | $display(C)$ | |
| $lock - X(B)$ | | |
| | | $grant - X(B, T_1)$ |
| $read(B)$ | | |
| $B := B + 100$ | | |
| $write(B)$ | | |
| $unlock(B)$ | | |

# What is the value of $C$?

- When we run this execution the value of $C$ will be 400 (needs checking!)
- Thus this execution (properly locking and unlocking the values) is *not* equivalent to any serial execution, for both possible serial executions displayed $C$ as 500!
- What went wrong?
- $T_2$ read *wrong* value of $B$, before it was modified by $T_1$ while the value of $A$ was already modified
- Can something be done to prevent such schedule?
- We can; one possible answer is: two-phase transactions (see below)

# What did we learn?

- In the scheme we consider (but there are others!) there are two kinds of locks
- There is a compatibility matrix that controls locks granting
- If we are not careful, integrity of results will not be preserved

# Two-phase transactions



- We will need them when we discuss two-phase locking protocol
- In these transactions every lock precede every unlock. Thus once unlocking starts within a transaction, no new locking requests (from that transaction) are accepted
- Here is one such transaction:
  $T_1$
  $lock - X(A)$;
  $read(A)$;
  $A := A - 100$;
  $write(A)$;
  $lock - X(B)$;
  $read(B)$;
  $B := B + 100$;
  $write(B)$;
  $unlock(A)$;
  $unlock(B)$;
- Here all lock requests precede all unlock requests
- (We will formally define two-phase protocol below)

# But deadlocks may occur

- Let us also modify the second transaction (the one displaying $C$) moving all unlocking operations till the end
- Now let us look at this partial schedule for these transactions (only salient features shown):

| $T_1$ | $T_2$ |
|---|---|
| $lock-X(A)$ | |
| $read(A)$ | |
| $A := A - 100$ | |
| $write(A)$ | |
| | $lock-S(B)$ |
| | $read(B)$ |
| | $lock-S(A)$ |
| $lock-X(B)$ | |

- After line 7 transaction $T_2$ can not proceed because the shared lock on $A$ can not be granted - so it waits
- But at line 8, the exclusive lock on $B$ can not be granted to $T_1$, so it also waits!
- Thus we get a deadlock

# Locking protocols - formal definitions

- By a locking protocol we mean a policy (i.e. a set of rules) which determines the behavior of transactions w.r.t. locking and unlocking data items
- Such protocol restricts the number of acceptable schedules
- We will discuss the most commonly used protocol, called *two-phase locking* or *2PL* protocol of Eswaran, Gray, Lorie and Traiger
- We say that a schedule $S$ is *legal* under a given protocol $P$ if $S$ is a possible schedule for a set of transactions that follows the rules of that protocol

# Locking protocols - definitions, cont'd

- We say that a protocol *ensures* conflict serializability if every schedule legal under that protocol is conflict serializable
- We assign to a schedule $D$ its *precedence relation* $\rightarrow$
- Here $T \rightarrow T'$ if there is a data item $A$ so that $T$ has a lock mode $L_1$ on $A$, later on in schedule $S$ $T'$ holds a lock $L_2$ on $A$ and those locks are incompatible (i.e. the value of compatibility matrix on these locks (and $A$) is 0)
- We want the relation $\rightarrow$ associated with $S$ to be *acyclic*
- Because a schedule is serializable only if the graph of transactions (where edges are determined by $\rightarrow$) has no cycles

# Granting policy, possibility of starvation

- A general lock-granting policy is this: *Whenever transaction $T'$ requests a lock on an item $A$ and no other transaction $T$ holds a conflicting lock on $A$, then the request is granted*
- (By whom? by CC manager)
- But there is a possibility of *starvation*. See for yourself:
  - $T$ holds a shared lock on $A$. Another transaction, $T'$, requests an exclusive lock on $A$. CC mgr tells $T'$ to wait (i.e. does not grant the request)
  - Then, another transaction, $T''$ requests shared lock on $A$
  - $T''$ gets the shared lock, the request of $T'$ still hangs
  - $T$ releases the lock on $A$, but $T''$ still has it, $T'$ waits
  - Nothing prevents another transaction $T'''$ getting shared lock on $A$ . . .
  - Transaction $T'$ is *starved*
  - Obviously, starvation is undesirable

# Preventing starvation

- To prevent starvation we need to modify the granting policy
- Here is the modified policy. We grant a lock on $A$ to transaction $T''$ if
  - No other transaction holds a conflicting lock on $A$
  - There is no transaction that is waiting for a lock on $A$ and which is waiting
- In our example, when $T''$ makes its request we tell it to wait
- That is the request of $T'$ will be granted before the lock of $A$ will be given to $T''$
- Thus, if we follow this policy, starvation will not occur

# Two-phase locking (2PL) protocol



- In this protocol we control locking and unlocking by enforcing two phases in a transaction:
  - In the *growing phase* a transaction acquires locks, but not releases them
  - In the *shrinking phase* a transaction releases locks but does not obtain any new locks

# 2PL protocol, cont'd



- The following transaction follows 2PL:
  $lock - X(A)$;
  $read(A)$;
  $A := A - 100$;
  $write(A)$;
  $lock - X(B)$;
  $read(B)$;
  $B := B + 100$;
  $write(B)$;
  $unlock(A)$;
  $unlock(B)$;
- We could switch lines 9 and 10, 2PL is preserved. But if we move line 9 after line 4, 2PL is not preserved
- Where else we could move line 9 preserving 2PL?

# More on 2PL



- If all transactions within a schedule follow 2PL then the schedule is serializable (i.e. its graph is acyclic)
- But 2PL does not protect from deadlock
- (The example of such 2PL schedule with deadlock was given above)
- 2PL does not protect from cascading rollbacks either
- A variation on 2PL, called *strict 2PL* is one in which exclusive locks are kept until the transaction commits. For instance transaction of previous page follows strict 2PL (s2PL)
- (Even stricter protocol, called *rigorous* 2PL, requiring *all* locks be kept till the end is also used)
- If all transactions within as schedule follow strict 2PL protocol then this schedule is both serializable and cascadeless
- Moreover for rigorous 2PL schedule *S* (i.e. all transactions within *S* follow rigorous 2PL protocol) the serial schedule equivalent to *S* is one following order of commits
- Both strict 2PL and rigorous 2PL are widely used in commercial DBMSes

# What did we learn?

- There is a variety of two-phase protocols
- They ensure strong properties of schedules, but at a cost
- These protocols are used in real DBMSes

# Lock conversion

- We can improve on schedules using *lock conversion*
- The idea is that we can, sometimes, upgrade from shared lock to exclusive lock
- Let us look at an example of two transactions

| $T$ | $T'$ |
|---|---|
| $read(A_1)$ | $read(A_1)$ |
| $read(A_2)$ | $read(A_2)$ |
| $read(A_3)$ | $C := A_1 + A_2$ |
| $write(A_1)$ | $display(C)$ |

- (There could be more stuff in these transactions, salient features shown only)
- When $T$ starts, in principle, it should request exclusive lock on $A_1$. So $T'$ would have to wait till $T$ ends
- But we really did not do this (until line 4) so we asked for stronger lock later

# Sharing and converting locks



- We can initially give $T$ *shared* lock on $A_1$ and upgrade it to exclusive when needed
- We need to be careful, though

| | |
|---|---|
| $lock-S(A_1)$ | |
| $read(A_1)$ | |
| | $lock-S(A_1)$ |
| | $read(A_1)$ |
| $lock-S(A_2)$ | |
| $read(A_2)$ | |
| | $lock-S(A_2)$ |
| | $read(A_2)$ |
| | $C := A_1 + A_2$ |
| | $display(C)$ |
| | $unlock(A_1)$ |
| | $unlock(A_2)$ |
| $lock-S(A_3)$ | |
| $read(A_3)$ | |
| $upgrade(A_1)$ | |

- We could upgrade $T$ on $A$ because $T'$ completed and released locks. In fact this schedule is equivalent to serial schedule $T'T$

# Approximate locking

- Here is the way *some* DBMSes handle locking requests
  - When a transaction $T$ issues a $read(A)$ request, the system puts the $lock - S(A)$ followed by $T : read(A)$ into the schedule
  - When a transaction $T$ issues $write(A)$ request, the system checks if $T$ already holds $lock - S$ on $A$. If so, the system issues $T : upgrade(A)$, followed by $T : write(A)$. Otherwise, the system issues $T : lock - X(A)$ and then $T : write(A)$
  - All locks held by a transaction are released when the transaction commits or aborts
- Clearly, while simple, this method does *not* ensure isolation

# Implementing locking

- Here is how one implements locking
- We assume that data items and transactions have unique identifiers
- We have a hash table on data items (thus there may be (and likely will be) collisions)
- Each hash value points to a linked list of data items (with that hash value)
- The reason why we have linked list of data items is to speed up access
- Each data item in that list has its own linked list of transactions having or requesting a lock on that data item

# Implementing locking, cont'd

- For each transaction in the list we have a bit indicating if the request was granted (we may need more bits for indicating the kind of the lock request)
- When a message with a request from a transaction $T_i$ for a lock of a specific data item $A$ comes, we put the identifier of that transaction at the end of the list for $A$
- (This means that the requests are handled in the order they are received and there will be no starvation)
- If the item is not locked, the request is granted

# Implementing locking, cont'd



- But if the item *is* locked, the lock is granted *only* if it is compatible with all the earlier locks in the list
- When an unlock message (for $A$) is received from a transaction $T$, then we eliminate $T$ from the list associate to $A$ we test the first request on the list that is not yet granted (if any exists) for compatibility with those which are currently granted. If it is compatible, it is granted (the case that no request is current is, of course, possible), otherwise it waits
- If a transaction (say $T'$) aborts all its waiting requests are deleted. When the effects of $T'$ are rolled back, all locks held by $T'$ are removed

# What did we learn?

- Locking is a delicate matter and sometimes is not implemented correctly (but is “good enough”)
- A natural data structure (hash table for fast access, then linked lists) can be used to implement locking

# Deadlock - definition

- A system is in a *deadlock state* if there is a set of transactions $K$ so that for any transaction $T \in K$ there is a transaction $T' \in K$ so that $T$ waits for $T'$ (i.e. for some data item $A$, $T$ waits for $T'$ releasing $A$)
- That is, in the waiting graph of the schedule there is a cycle
- Hence, none of transactions on that cycle can make a progress
- The issue is how can we deal with the deadlock
- There are two possibilities:
  - One is *deadlock prevention* (i.e. making sure we never get into a deadlock state)
  - The other one is that we do not prevent, but we are able to *detect* a deadlock and *recover* from it

# Deadlock, cont'd

- Whatever technique we use, some transactions may have to be rolled back, either as part of prevention, or recovery
- The system maintains statistics on the performance, length of transactions, throughput, etc.
- If the statistics indicates that likelihood of getting into deadlock is significant, prevention is better
- But if the chance of getting into deadlock is small, it is better to detect and recover
- (But there is no free lunch - statistics is costly)

# Deadlock prevention

- Two possible approaches:
  - We could order requests for locks (they come in some order) to make sure an earlier request wins
  - We could request all locks for a transaction at once and grant them all or not at all
- The simplest thing is to get locks on all data a transaction needs (or may need) - the second solution
- But there are problems:
  - It may be difficult to predict what data will be needed
  - (For instance: we want to find all customers who placed orders bigger than the average order in the least affluent zip code. We do not know when the transaction starts what will be that least affluent zip code and its average transaction)
  - The data-item utilization may be too low (we over lock the data)

# Deadlock prevention, preemption

- The idea is that transactions get *timestamps*. That is we order the transactions according to their starting points
- When a transaction $T_i$ holds a lock on data item $A$ and $T_j$ requests a lock we need to decide what is going to happen (one of them could wait while other progresses, or one of them could roll back and the conflicting request disappears one way or another)

# Deadlock prevention, preemption, cont'd

- Two specific schemes were proposed:
  - *Wait-die* (non preemptive) If $T_i$ requests data held by $T_j$, $T_i$ is allowed to wait *only* if $i < j$ (i.e. $T_i$ started earlier). Otherwise, $T_i$ rolls back
  - Example: we have three transactions $T_{10}, T_{15}$, and $T_{20}$. When $T_{10}$ requests data held by $T_{15}$, then $T_{10}$ waits. But if $T_{20}$ requests data held by $T_{10}$ then $T_{20}$ rolls back and gets back to the queue
  - *Wound-wait* (preemptive) If $T_i$ requests a data item held by $T_j$ then$T_i$ is allowed to wait *only* if $j < i$. Otherwise $T_j$ rolls back
  - Example: again we have three transactions $T_{10}, T_{15}$, and $T_{20}$. When $T_{10}$ wants data item held by $T_{15}$ then $T_{10}$ gets it, $T_{15}$ is rolled back. When $T_{20}$ wants data item held by $T_{15}$, $T_{20}$ waits
- Many rollbacks may occur, throughput suffering

# Limited wait

- Yet another possibility is to prespecify the waiting time
- When a transaction waits too long - it is rolled back
- Easy to implement (obviously)
- But what would be that wait time?
- This solution works well then transactions are short!
- But in general it is impossible to predict the length of transactions
- Moreover, in this approach starvation may occur, if a transaction has to wait many times and be rolled back repeatedly

# Deadlock detection

- Clearly, it is a graph-theoretic problem, for all we care is detection of presence of cycles (but where?)
- Formally, we define a *wait-for graph* for a schedule, $G_{wf}$
- This is a directed graph; vertices are transactions, edges are pairs $(T_i, T_j)$ where for some data item $A$, $T_j$ has a lock on $A$ and $T_i$ seeks an incompatible lock on $A$
- Deadlock (as stated above) is presence of a cycle in $G_{wf}$
- Example: $T_2$ waits for $T_{15}$ which waits for $T_{23}$ which waits for $T_2$. The data items involved may be different or not
- Of course, as the schedule is executed, the graph $G_{wf}$ changes
- When $T_i$ requests a lock on an item $A$ held by $T_j$ an edge $T_i \rightarrow T_j$ is added
- And when $T_j$ releases the lock, the edge is erased
- (There are interesting issues when this graph is processed)

# Deadlock detection, cont'd



- Deadlock detection (i.e. cycle finding) costs. Thus the issue raises: when should it be activated?
- Actually, depends on statistics; more deadlocks, more searches for deadlock
- Once you detect a deadlock, there may be more than one simple cycle
- Which should be broken first?
- How should we select the "victim" i.e. a transaction on a cycle?

# Selecting a victim

- It is more complex than it looks, for it is not enough to look at the complexity of transactions
- Here are some factors:
  - How long transaction computed and how close to completion
  - How many data items a transaction already used and how many it needs for completion
  - How many transactions may be involved in the rollback (for there may be cascading rollbacks)

# Rollbacks, possibility of starvation

- Complete, i.e. entire transaction rolled back
- Partial, only some actions undone
- But if it is so, then the recovery system needs to be able to do partial rollbacks and test for presence of cycles after partial rollbacks
- This significantly increases the cost and complexity of recovery
- Also when we have repeated rollbacks, starvation may occur

# What did we learn?

- Deadlock occurs and needs to be handled
- There are various ways to handling deadlock
- Deadlock handling always requires a rollback

# Timestamp ordering protocol - introduction

- (Previously we discussed lock-based protocols)
- Now, the idea is to determine the serializability order by looking at the order in which transactions were submitted
- To have such total order, we assign to transactions their *timestamps*
  - One way to do this is to assign to a transaction the system time of submission
  - Another technique is to assign a “serial number”, i.e. the counter value
- **TS**($T$) - the timestamp of $T$

# Timestamps on a data item



- Timestamps are on transactions, but we can use them for marking data items
- Given a data item $A$ we assign to $A$ two timestamp values:
  - $R - timestamp(A)$ – the maximum of timestamps of transaction that read $A$
  - $W - timestamp(A)$ – the maximum of timestamps of transaction that write $A$
- These values are dynamic; they change each time $A$ is read or written

# Timestamp ordering protocol



- (The idea is to create schedule that is conflict equivalent to the serial schedule corresponding to the order of timestamps of transactions)
- When $T_i$ submits $\mathrm{read}(A)$ then
  - (a) If $\mathbf{TS}(T_i) < W - timestamp(A)$, then $T_i : \mathrm{read}(A)$ is rejected and $T_i$ is rolled back
  - (b) If $\mathbf{TS}(T_i) \geq W - timestamp(A)$, then $T_i : \mathrm{read}(A)$ is placed in the schedule and the value of $R - timestamp(A)$ is updated to $\max(\mathbf{TS}(T_i), R - timestamp(A))$
- Justification of (a): When $\mathbf{TS}(T_i) < W - timestamp(A)$ then $T_i$ asks to read value of $A$ modified by a later transaction. This is potentially an error, needs to be rejected, and $T_i$ must be rolled back and restarted
- Justification of (b): $T_i$ wants to read correct value. But once it does, the read-timestamp of $A$ changes and so we need to update it

# Timestamp ordering protocol, cont'd



- When $T_i$ submits $\mathrm{write}(A)$
  - (a) If $\mathbf{TS}(T_i) < R - timestamp(A)$, then the request is rejected, $T_i$ rolled back
  - (b) If $\mathbf{TS}(T_i) < W - timestamp(A)$ then the request is rejected, $T_i$ rolled back
  - (c) If $\mathbf{TS}(T_i) \geq R - timestamp(A)$ and $\mathbf{TS}(T_i) \geq W - timestamp(A)$ then the command $T_i : \mathrm{write}(A)$ is placed in the schedule and the value $W - timestamp(A)$ is updated to $\mathbf{TS}(T_i)$

# Timestamp ordering protocol, cont'd

- When a transaction is rolled back it comes back to the queue with *new* (larger) timestamp
- Justification of (a): The value that $T_i : \mathrm{write}(A)$ is writing was needed earlier (because some transaction with the smaller timestamp was reading $A$, not realizing $T_i$ will overwrite it.) We have to roll back $T_i$
- Justification of (b): The value of $A$ that $T_i$ is producing overwrites value of $A$ which should be produced later. We have to roll back $T_i$
- Justification of (c): There are no irregularities, command is accepted. However the value $W - timestamp(A)$ changes (goes up) and needs to be updated
- We observe that potentially we err on the "safe side", maybe this write is harmless

# Example

- Familiar example, $T_1$: "sum of two values is displayed", $T_2$: "*marek* transfers to *dexter* $100"

| $T_1$ | $T_2$ |
|---|---|
| read($A$) | |
| | read($A$) |
| | $A := A - 100$ |
| | write($A$) |
| read($B$) | |
| | read($B$) |
| $C := A + B$ | |
| *display*($C$) | |
| | $B = B + 100$ |
| | write($B$) |

- After line 1: $R - timestamp(A) == 1$, but $W - timestamp(A) == 0$
- After line 2: $R - timestamp(A) == 1$, but $W - timestamp(A) == 0$
- After line 4: $W - timestamp(A) == 2$ (so $T_1$ will not be permitted to read $A$ - if it wanted to)
- After line 5: $R - timestamp(B) == 1$
- After line 6: $R - timestamp(B) == 2$
- After line 9: $W - timestamp(B) == 2$
- What is interesting in this example is that if we change stamps of $T_1$ to $T_2$ and conversely, this is not a valid schedule according to timestamp ordering protocol

# Timestamp ordering protocol, cont'd



- Deadlock is not possible under this protocol (because there is no waiting)
- Starvation is possible (because a transaction may be sent back to the queue repeatedly). If this happens we may need to force schedule to allow $T_i$ execute
- Recoverability and cascadeless properties of schedule are not assured although we get them if we modify the protocol so all writes are at the end of transaction (thus at the cost of forcing other transactions to roll back)

# What did we learn?

- There are alternatives to 2PL
- By using timestamps we can force serializability, too
- As in the case of 2PL there are variations on the timestamp ordering protocol

# Snapshot isolation - some details

- Widely used throughout the industry, for instance in Oracle
- Recall that snapshot isolation means providing a transaction $T$ with a "snapshot" of database and allowing $T$ to work on that snapshot
- Snapshot contains only states of data written by *committed* transaction. Thus if a transaction $T$ works on an item $A$ that was written by $T'$ which was not committed, then $T$ sees the state of $A$ from before $T'$ started. Seems strange *but* if $T'$ aborts then rollback of $T$ prevented
- This means we need to know which transaction wrote what etc. but this is easy to implement

# Snapshot isolation, cont'd

- Under SI, read-only transactions never wait, and are never aborted
- There are, normally, many such transactions (SELECT transactions are of this form) so this speeds up processing
- Observe that with snapshot, read-only transactions result in answers on the database as it was when transaction started
- Thus their execution does not affect serializability
- With SI, all updates are made when the transaction is partially committed, i.e. all operations done
- The *commit* for SI is *atomic*; either all writes are done or none at all

# The phenomenon of *lost update*

- The issue is whether to allow a transaction to commit or not
- In principle two different transactions could attempt to update same data item
- But these transactions computed in isolation. If both are allowed to write same item, the second one to execute will overwrite the result of the first!
- This phenomenon is called *lost update*
- Think about *dexter* getting $100 from *marek* and then getting interest on his balance. The lost update would mean that *dexter* balance is incorrect (we discussed this example before, but in a different context)
- Lost updates must be prevented

# Policies for preventing lost updates



- Two policies used to prevent lost updates
  - *First commiter wins*
  - *First updater wins*
- To analyze the situation we need the language. We call a transaction $T'$ *concurrent* to $T$ if $T'$ is active or partially committed at any point from the start up to and including the point when the test is made

# *First commiter wins* policy

- When a transaction $T$ enters a partially committed state (i.e. all it needs to do is to commit)
  - The system tests if any transaction $T'$ concurrent with $T$ already wrote an update to the database on *some* item $A$ that $T$ intends to write (i.e. $T$ would execute a lost update on $A$)
  - If such $A$ exists, $T$ aborts (and thus prevents lost update)
  - Else $T$ commits and executes its updates
- The reason we call this policy *first commiter wins* is that if $T$ and $T'$ conflict then the first one that tests the conditions (and thus committed) wins

# *First updater wins* policy

- This strategy uses locks (but locks apply only to *write* operations, because we process in isolation)
- When a transaction $T$ intends to write an item $A$, it applies for a lock
- If such lock is not already held by another transaction $T'$ the following happens:
  - If the item $A$ was updated by a transaction $T'$ which is concurrent with $T$ then the transaction $T$ is aborted (to prevent the lost update)
  - Else, $T$ may proceed with its execution (thus the isolation is not complete!)

# *First updater wins* policy, cont'd



- If, however, some transaction $T'$ holds already a lock on $A$, $T$ can not proceed and $T$ waits till $T'$ commits or aborts
  - If $T'$ aborts, the lock is released and $T$ can obtain the lock (but careful here, there may be more transactions waiting for that lock). After that lock is acquired the same test (Was a concurrent transaction writing $A$?) is performed. If so, $T$ aborts (to prevent lost update). If not, $T$ proceeds
  - Else, i.e. if $T'$ commits, $T$ aborts

# *First updater wins* policy, cont'd

- Locks held by $T$ are released if $T$ terminates (i.e. commits or aborts)
- This approach is called *first updater wins* because when transactions conflict, the first one that established a lock is allowed to commit
- Thus the first to lock (thus update) is allowed to proceed with the updates and "wins"

# Snapshot isolation, cont'd

- Snapshot isolation is attractive because (with appropriate data structures) the overhead is low
- There are no aborts unless two concurrent transactions update same data item
- For that reason modern DBMSes, specifically Oracle, use it even at the serializability level
- But there are several problems with SI, specifically consistency check, enforcing of UNIQUE (thus also PRIMARY KEY) constraints, and of FOREIGN KEY constraints

# Problems with consistency



- Say, *marek* has two bank accounts: checking and saving
- The integrity constraint is that the sum of amounts is positive
- Now, *marek* has $300 in checking, $400 in savings
- *marek* takes $500 from each account
- Each of two transactions, in parallel, checks the consistency
- Each finds consistent state after execution
- But if both executed the sum of balances is -$300

# Problems with consistency, cont'd



- This phenomenon is called *write skew*
- We can establish that write skew exists when we formally analyze the precedence graph nad find that it is cyclical
- To prevent such situation SQL provides a language to check for integrity constraints and prevent write skew
- The qualifier is *for update*

# Other problems



- Clearly, concurrent transactions may insert two records with same key
- (For they checked the snapshot at the beginning of the transactions, not at the end)
- The UNIQUE constraint can be similarly violated
- Likewise there may be problems with FOREIGN KEY (what if a record with referenced value disappeared?)
- For that reason the consistency check may be done (as with *for update* qualifier) at the end

# What did we learn?

- Snapshot Isolation is a widely used and attractive technique
- There are two fundamental policies to enforce absence of lost updates
- Snapshot Isolation, though attractive, has its problems and requires care - if used

# But database lives ...

- So far we tacitly assumed that no new data items (records, objects) appear and no existing data items are deleted
- But in reality new data items come in and old data items disappear
- Thus transactions may involve instructions of the form $\mathrm{delete}(A)$ and $\mathrm{insert}(A)$
- We need to see how the concurrency-handling protocols deal with such instructions

# Semantics of new operations



- The meaning of $\mathrm{delete}(A)$ is: “delete the data item $A$ if it is in the database or report an error if $A$ is not in the database”
- The meaning of $\mathrm{insert}(A)$ is: “insert the data item $A$ if it is not in the database or report an error if $A$ is in the database”
- We have to be a bit careful, because the statement “$A$ is in the database” must be made precise
- But if we work on the granularity level of records, then the situation is clear, we have or do not have a record with a given key value

# Conflicts in the new situation, deletes

- As before instructions involving different data items do not conflict
- Let us first assume the instruction $I$ is $\mathrm{delete}(A)$, and is part of transaction $T$, the instruction $I'$ is part of transaction $T'$
  - When $I'$ is $\mathrm{read}(A)$. Then $I$ and $I'$ are in conflict. If $T$ is submitted before $T'$ then the logical error occurs in transaction $T'$ (so it will be rolled back). If $T'$ is submitted before $T$, $T'$ executes $I'$ successfully
  - When $I'$ is $\mathrm{write}(A)$. Then $I$ and $I'$ are in conflict. If $T$ is is submitted before $T'$ then the logical error will occur in $T'$ because $T'$ attempts to modify an nonexisting object. Thus $T'$ fails, $T'$ is rolled back. If $I'$ is submitted before $T$, $T'$ executes $I'$ successfully

# Conflicts, deletes, cont'd

- We continue conflict description for $\mathrm{delete}(A)$
  - When $I'$ is $\mathrm{delete}(A)$. Then $I$, $I'$ are in conflict. A logical error occurs in one of transactions depending on the order of operations in the schedule. If $I$ executed *before* $I'$ then the error will be in $T'$ because $T'$ executes delete of non-existing object. Otherwise the error is in $T$. The transaction with logical error will be rolled back
  - $I'$ is $\mathrm{insert}(A)$. Then $I$, $I'$ are in conflict. We need to consider two cases
    - If the item $A$ did not exist before $T$ and $T'$ started then if $I$ comes before $I'$ then $T$ will have logical error and will be rolled back because it attempts to erase a non-existing item. Otherwise $T'$ executes its insert, and then $T$ its delete
    - If the item $A$ did exist before $T$, $T'$ started, then with $I$ coming before $I'$ execution is normal ($T$ erases $A$„ then $T'$ inserts $A$ back) and if the order is opposite, $T'$ rolls back because it attempts to insert an existing object

# Deletes under 2PL and TS protocols

- Under 2PL an exclusive lock on an item $A$ is needed *before* $\mathrm{delete}(A)$ is executed
- Under timestamp protocol tests are performed before $\mathrm{delete}(A)$ is executed. Assume $T$ issues $\mathrm{delete}(A)$
  - If $\mathbf{TS}(T) < R - timestamp(A)$, then $T$ is rolled back. The reason is that some later transaction read $A$ and so if we delete $A$ it read an non-existing object
  - If $\mathbf{TS}(T) < W - timestamp(A)$, then $T$ rolls back. The reason is that some later transaction wrote $A$ and so if we delete $A$ it wrote a non-existing object
  - Otherwise $T : \mathrm{delete}(A)$ is executed and $T$ proceeds

# Conflicts in the new situation, inserts



- As before $\mathrm{insert}(A)$ conflicts with all other operations (why?)
- $\mathrm{read}(A)$ and $\mathrm{write}(A)$ will result in a logical error if executed before $\mathrm{insert}(A)$
- Clearly, $\mathrm{insert}(A)$ is very similar to $\mathrm{write}(A)$, as inserting, in particular implies writing

# Inserts under 2PL and TS protocols



- Under 2PL, if $T$ issues $\text{insert}(A)$ then
  - If $A$ does not exist then $T$ gets an exclusive lock on $A$ and proceeds
  - Otherwise (that is if $A$ exists), $T$ gets a logical error and is rolled back
- Under timestamp protocol
  - If $A$ does not exist then $T$ executes $\text{insert}(A)$
  - Otherwise (that is if $A$ exists), $T$ gets a logical error and is rolled back

# What did we learn?

- Databases live, and insert and delete operations occur
- We often think about updates as “delete and insert”, but this is not what really occurs
- The presence of deletes and inserts complicates concurrency control protocols, but those can be adjusted to the new situation

# Phantom phenomena

- We have two SQL commands and the corresponding transactions
- The first one is a read-only select query:
  SELECT SUM(balance) AS "Total Assets"
  FROM *accounts*
  WHERE location = “lexington”;
- The second one is an insert query:
  INSERT INTO *accounts*
  VALUES (043, marek, lexington, 149.99)
- We now have transactions $T$ and $T'$ corresponding to these queries
- What about schedules involving these transactions?

# Phantom phenomena, cont'd



- The issue is whether marek's balance is included in the answer to the first query, or not
- There is a potential for conflict because:
  - If marek's balance is included in the answer, then $T'$ should follow $T$ (in a serial execution)
  - If marek's balance is not included then the order is opposite
- This second case is funny because the order is entailed *even though $T$ and $T'$ do not share access to any record*!
- Thus there is a conflict in this case

# Phantom phenomena, cont'd



- But there are other phenomena of this sort
- Here is one:
  - Client dexter lives in Louisville, but now he moves to Lexington
  - We do not insert anything, we just change the value of the attribute *location* in dexter's record
- So what about the balance of Lexington clients?
- In other words, does dexter contribute to the Total Assets in Louisville? In Lexington?

# Dealing with phantom phenomena

- Clearly something needs to be locked, but what?
- Recall that in SQL databases *every* table possesses an index that consists of pages organized in ISAM or B+ tree, or a hash index
- That index is on PRIMARY KEY
- Let us assume we have a tree-like index
- (We may have more than one index on a given table, but it is costly)
- The idea is to lock some parts of the index *and* require that some locks be obtained on various leaf-pages of one or more indices

# Dealing with phantom phenomena, cont'd

- For the SELECT queries (i.e. look-up) we will get shared locks
- But for insert, delete and update queries we will have to obtain exclusive locks on (parts of) the index (or indices, if we maintain more than one)
- We will follow the rules on compatibility of locks as studied above
- We will need a protocol for handling such locks
- The fundamental idea here is: *Deal with the index* before *you deal with the table*

# Index-locking protocol

- Here are the steps:
  1. A transaction can access records of a table *only* after it locates them through indices
  2. A transaction that performs a look-up obtains a *shared lock* on the leaf nodes it accesses
  3. A transaction can not insert, delete or update a record in a table without updating all indices of that table. Such transaction must obtain *exclusive lock* on all leaves that are affected
     - 3.1 For insert it must obtain X-locks on all nodes that contain items *after* insertion
     - 3.2 For delete it must obtain X-locks on all nodes that contain items *before* deletion
     - 3.3 For update it must obtain X-locks on nodes that contain items both before and after updates
  4. 2PL protocol is followed for locks

# What this protocol does in our examples?

- In the first example if the first transaction started first, it gets a shared lock on the leaf nodes of the index for *accounts*
- When the second transaction wants to insert, it has to get an exclusive lock on nodes containing records with location *lexington*
- But then this second transaction has to wait until these locks are released
- We can also similarly analyze the requests of the second transaction
- In the other transaction we lock nodes containing records with the value of location *louisville* and with value of location *lexington*
- In both cases we eliminated phantom phenomena
- There is a cost involved (indices have to be maintained!)

# What did we learn?

- Phantom phenomena are quite real and if not controlled, result in compromising integrity of query processing
- We can handle phantom phenomena, but at a cost of maintaining indices (extra space, and extra work is needed for index maintenance)
- A form of 2PL protocol (but involving index leaf pages) is used for handling phantom phenomena

# Final remarks

- Transactions processing is the essence of DBMS
- SQL provides an important “syntactic sugar” seen by ordinary users, but the real action is in the layer below the level of SQL
- Understanding of DBMS goes significantly beyond SQL
- Current versions of SQL provide support for various issues related to transactions